

Remote access and the machine

The four verbs that change Windows rather than SD, and the one thing they all require.

Most administrator verbs change something inside SD. These four change the machine SD is running on: whether an ssh server exists, who may reach it, whether SD opens its API socket, and whether `sd` runs from any directory.

<code>ssh.server</code>	adds or removes the Windows OpenSSH server
<code>remote.ssh</code>	decides who may reach it
<code>remote.api</code>	decides whether SD opens its API socket, and who may reach it
<code>append.sd.path</code>	puts SD's program directory on the Windows system PATH, or takes it off

None of these gives an administrator remote access, and no setting does. Opening ssh or the API to other computers opens it for ordinary and programmer accounts. **An administrator account is refused a session from any machine but this one**, over either route, after its password has been checked — so a route you have just opened will still turn your own administrator account away if you try it from your laptop. That is deliberate: administration needs a session Windows can show a consent prompt on, which means the console, or a remote desktop or remote-control product installed as a service. The operating system access page covers why the tier is treated this way.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

All four need an elevated session, including to report

Each of them begins by testing the administrator flag, and stops if it is not set:

```
:ssh.server
Command requires administrator privileges
```

That was an administrator account. Holding the administrator tier is not enough — **the session itself has to be elevated**, and the test runs before the keyword is read, so the reporting forms are refused too.

The reason is the work they do rather than the information they give. Each one reaches Windows through SD's elevated helper, and an unelevated session has no helper to reach it through. Reading the answer costs the same rights as changing it: `Get-WindowsCapability` cannot report whether a Windows capability is installed without administrator rights either.

An administrator gets an elevated session by signing in at the console as a Windows administrator, which elevates at login. A session that reached an ordinary account by `logto` does not have one, and cannot get one — the first `logto` out of the system account ends elevation deliberately, which is covered under operating system access in this set.

Every one of them reports when given no keyword

```
ssh.server
remote.ssh
remote.api
append.sd.path
```

With no keyword each reports the current state and changes nothing. This is the supported way to answer "how is this machine set up?", and for ssh it is the only reliable way — see below.

ssh.server

```
ssh.server {install | remove}
```

FINDING OUT WHETHER A SERVER IS INSTALLED

Windows 11 does not list the OpenSSH server in Settings → Apps, and it has never appeared in Control Panel → Programs and Features. Somebody looking there concludes it was never installed, which is the wrong conclusion and an easy one to reach.

It is a Feature on Demand, not a program. In Windows 11 it lives under **Settings → System → Optional features**; in Windows 10 it was under Settings → Apps, which is where anyone who learned this on Windows 10 will look first.

The PowerShell route is not a fallback either, because `Get-WindowsCapability -Online` needs administrator rights even to read.

So `ssh.server` with no keyword is the answer: it reports, changes nothing, and works because it goes through SD's elevated helper.

```
The OpenSSH server is installed and running.
```

Who may reach it is a separate setting - use "remote.ssh" to see or change it.

INSTALL

The capability comes from Windows Update. It can take several minutes, and up to about an hour on a slow machine or connection, with little on screen while it runs. The verb asks before starting for that reason.

Afterwards Windows needs a restart before the service exists, so no SD account can sign in until the machine has been restarted.

REMOVE

`remove` asks first, and the question is not a formality. Accounts SD creates sign in over ssh or over the API and are denied the console and Remote Desktop. For an account that has only ssh — including at this keyboard, where it arrives by `ssh localhost` — removing the server takes away its only way in.

The removal is staged behind a reboot. Windows reports success while `sshd.exe` is still on disk and the service is still running, so ssh continuing to work afterwards is expected and is not a fault.

One consequence is worth knowing before you remove it. `C:\ProgramData\ssh` is left in place, because Windows does not remove it with the capability. It holds the host keys and `sshd_config`. That matters for one thing: running the SD installer on this machine again. Setup compares `sshd_config` against the copy Windows ships, `sshd_config_default` — and that copy went with the capability, so Setup cannot tell whether the configuration has been edited and stops rather than guess.

`ssh.server install` is not affected: it puts the server back and `sshd_config_default` comes back with it. Remove the directory as well only if you do not intend to run an ssh server on this machine again.

remote.ssh

```
remote.ssh {on | off}
```

`on`

other computers on the network may connect over ssh

`off`

only this computer may connect

It takes effect at once, needs no restart, and disconnects nobody.

`off` **does not stop the server.** It scopes the Windows firewall rule `OpenSSH-Server-In-TCP` rather than disabling it, and that distinction is load-bearing: disabling the rule would also break `ssh localhost`, which is how an SD account signs in at this keyboard. To remove the server itself, use `ssh.server remove`.

If there is no ssh server, the verb says so and points at `ssh.server install` rather than reporting a state it cannot have.

remote.api

```
remote.api {on | local | off}
```

on	SD listens on port 4243 and other computers may connect
----	---

local	SD listens on port 4243 and only this computer may connect
-------	--

off	SD opens no API socket at all
-----	-------------------------------

There are two axes here and the verb sets both. `APIPORT` in `sd.conf` decides whether SD opens a socket at all; the firewall rule decides who may reach it. `local` exists because they are separate — listener up, firewall shut.

That is also why the three keywords behave differently:

on	and	off	change whether SD opens a socket, which it decides when it starts. They offer to restart SD
----	-----	-----	---

local	and	on	differ only in the firewall when the listener is already up, and take effect at once
-------	-----	----	--

The restart ends every SD session, including the one that asked for it. The verb says so in the question rather than afterwards. Declining leaves the setting written and it takes effect when SD is next started.

If the listener is set but the firewall rule cannot be written, the verb says which half succeeded and prints the two commands that set the rule by hand. This is a real state and not a theoretical one: the setting and the rule are different objects and either can fail on its own.

append.sd.path

```
append.sd.path {on | off}
```

on	put SD's program directory on the Windows system PATH
----	---

off	take it off
-----	-------------

This is what lets `sd` run from any directory rather than only its own.

It is the system PATH, not the user's. The value lives under `HKLM\...\Session Manager\Environment`, so the change applies to every account on the machine. A terminal that is already open keeps the PATH it started with; open a new one to see the change.

Turning it off does not make SD unreachable. `sd` still runs from its own directory or by full path, and `append.sd.path` `on` puts it back.

WHY THIS VERB EXISTS

It is not symmetry with the other three. The installer's PATH tickbox could set this once and never again: nothing removed the entry except the uninstaller, so clearing the box on a later run did nothing at all. An upgrade shows no tasks page whatsoever, on the principle that an administrator who wants to make further choices has command-line tools for them — and that left the PATH setting with no way to be changed after the first install. This is that tool.

WHY IT IS NOT CALLED SET.PATH

Inside SD, *path* already means the **account's** path: `pathname` is a VOC keyword, `where` prints the account pathname, and `@path` holds it. `set.path` would read as setting that. `os.path` collides with the notion of a host filesystem path. `append.sd.path` cannot be read as either.

When a change fails

All four report a failure rather than falling silent, and every failure message names what did not happen and confirms that nothing was changed. Two status values recur and mean different things:

status 1	Windows refused the change, or SD is not running elevated
----------	---

status -1	the helper could not be run at all
-----------	------------------------------------

`remote.api` adds status 2, which means `sd.conf` could not be read — missing, unreadable, or carrying no `APIPORT` line to recognise.